

21IPE314P / RTOS Programming

Review – 2 Report

Submission Date: 17.4.2026

Title of the project: **Hardware-in-the-Loop infusion pump controller prototype using QNX Neutrino RTOS**

Student 1 Reg no. and Name and Dept: **Naga Praneeth Reddy, RA2311004010472, ECE-H**

Student 2 Reg no. and Name and Dept: **Rudra Shukla, RA2311004010482, ECE-H**

S.No.	Evaluation Criteria	Description	Max Marks	473	482
1	Design Implementation	Progress in converting design into working modules	2		
2	Module Development	Development of subsystems/components	1		
3	Algorithm / Methodology	Clarity in approach and techniques	2		
4	Tools & Technology	Use of hardware/software tools	1		
5	Intermediate Results	Initial outputs/testing	1		
6	Problem Solving	Debugging and issue handling	1		
7	Presentation	Clarity in explanation	2		
Total Marks			10		

Course Teachers: Dr. S. Dhanalakshmi and Dr. Jaba Deva Krupa A

1. Project Overview

This project implements a Hardware-in-the-Loop infusion pump controller prototype using QNX Neutrino RTOS on a Raspberry Pi 4B as the real-time controller, with an STM32 Nucleo-F401RE microcontroller acting as the hardware-side plant simulator. The two platforms communicate over a direct 3.3V UART link at 115200 baud, and all inter-process coordination within QNX is handled through the operating system's native message-passing mechanisms.

The motivation for choosing QNX as the target platform comes from its microkernel architecture and its suitability for mixed-criticality embedded applications. Rather than attempting to build a fully IEC 62304-certified implementation, this project is deliberately scoped as a safety-informed classroom prototype. The design priorities are a reliable hardware-validated communication path, a cleanly separated QNX process architecture, and demonstrable fault detection behaviour. These goals have been fully achieved through a sequence of incrementally validated implementation stages.

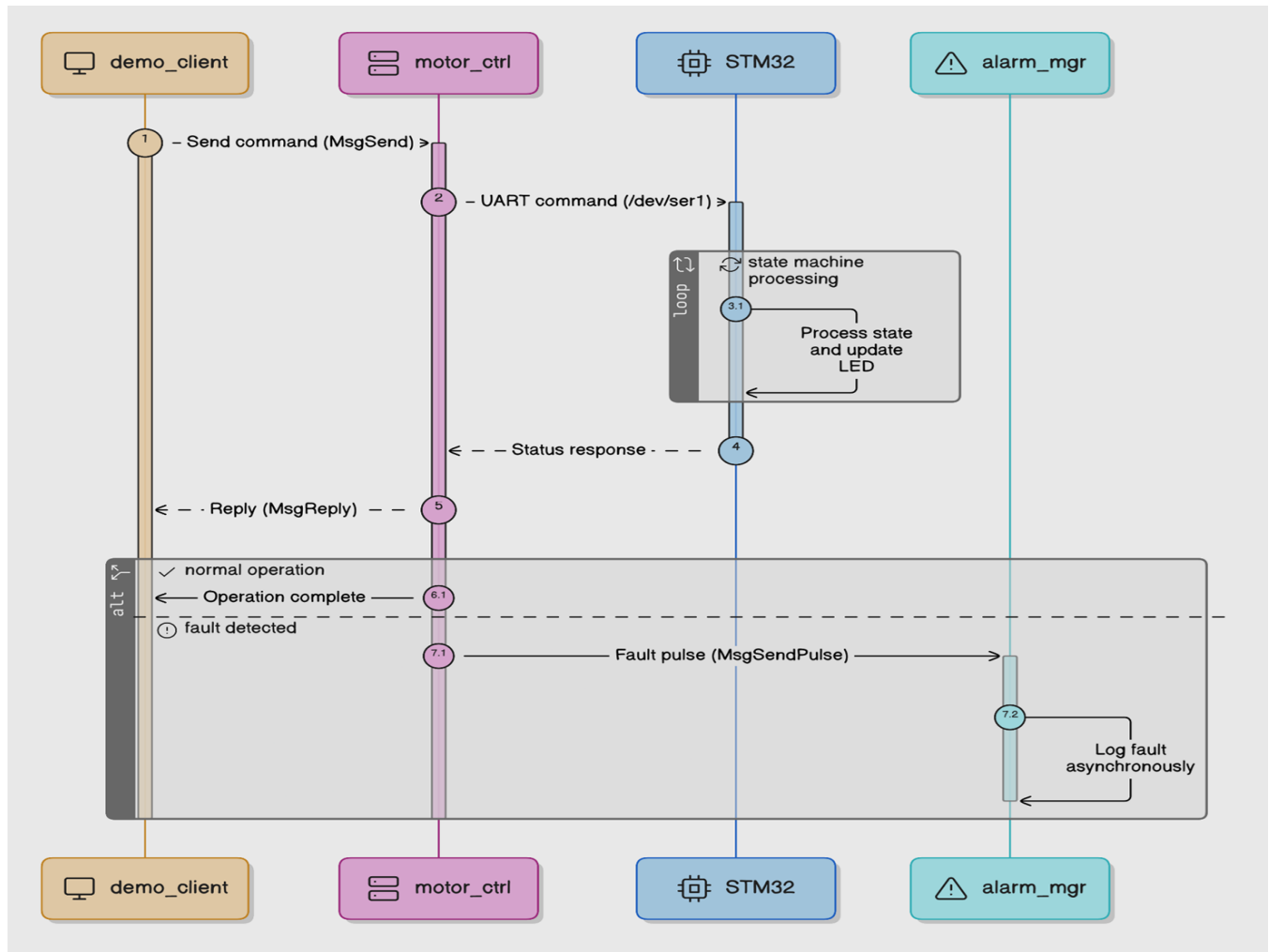
2. System Architecture

The system is structured around three QNX processes and one STM32 firmware component. On the QNX side, `motor_ctrl` acts as the central control server. It exclusively owns the `/dev/ser1` serial device and is the only process that communicates directly with the STM32. A small client application called `demo_client` sends `START`, `STOP`, and `STATUS` commands to `motor_ctrl` using `MsgSend`, and receives synchronous replies via `MsgReply`. A separate `alarm_mgr` process runs independently, waiting on its own channel for fault pulses that `motor_ctrl` delivers asynchronously using `MsgSendPulse`.

This design deliberately avoids implementing a custom QNX resource manager. A resource manager would have required significantly more development time for POSIX-interface boilerplate that adds no visible demo value. The message-passing server approach chosen here achieves the same goal of centralised UART ownership with a much simpler implementation, while still exercising the core QNX IPC primitives. The separation between synchronous command control and asynchronous fault notification is a key architectural feature: `demo_client` is never blocked waiting for an alarm, and `alarm_mgr` receives fault notifications independently of whether any client command is in flight.

The STM32 firmware runs a simple four-state machine covering `IDLE`, `RUNNING`, `FAULT`, and `EMERGENCY_STOP` transitions. The onboard LED provides continuous visual feedback of the current state: off for `IDLE`, solid on for `RUNNING`, and blinking

for FAULT. The user button on the Nucleo board serves as a manual fault injection source for demonstration purposes.



3. Protocol Design and Methodology

All UART communication between QNX and STM32 uses a fixed framing structure. Each packet consists of a sync byte (0xAA), a one-byte command ID, a one-byte payload length, an optional payload of up to 16 bytes, and a trailing CRC8 checksum. The CRC is computed over the command, length, and payload bytes using the CRC8-MAXIM (Dallas) algorithm in its LSB-first reflected form with polynomial 0x8C. The same CRC function is used identically on both the QNX and STM32 sides, ensuring that any byte corruption or framing error is detected at the receiver.

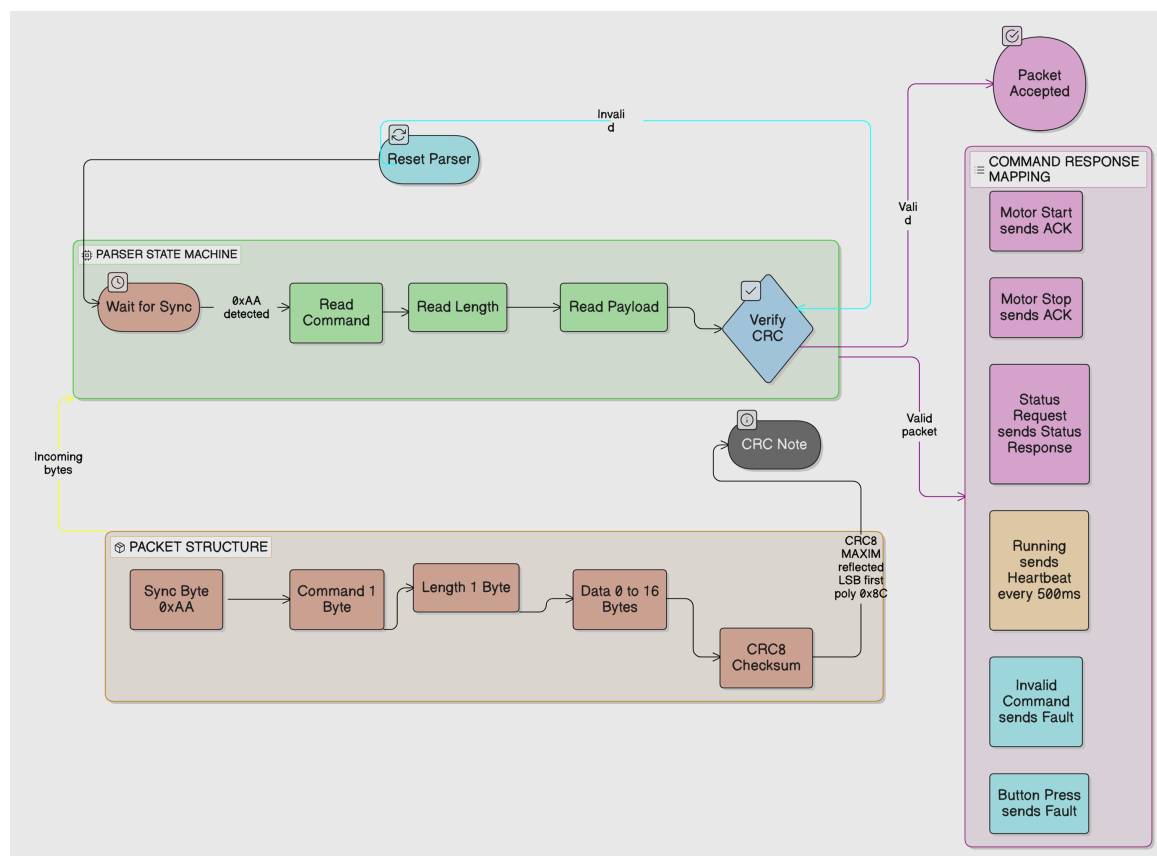
Packet parsing is handled by a byte-level state machine rather than attempting to read a complete packet in one read call. The parser moves through five states: waiting for the sync byte, reading the command, reading the length, accumulating payload bytes, and finally

verifying the CRC. This approach is more robust on serial links where reads may return fewer bytes than requested. When a CRC mismatch is detected, the parser resets to its initial state and returns an error code, which triggers a fault notification upstream.

The command set used we defined consists :-

- MOTOR_START (0x01)
- MOTOR_STOP (0x02)
- STATUS_REQ (0x03)
- HEARTBEAT (0x04)
- ACK (0x05)
- FAULT (0x06)
- STATUS_RSP (0x07).

The STM32 responds to START and STOP with an ACK packet, answers STATUS_REQ with a STATUS_RSP packet carrying a one-byte motor state value, sends HEARTBEAT packets at 500ms intervals while in RUNNING state, and sends a FAULT packet either in response to an invalid command or when the user button is pressed.



4. Module Development

Development was organised into discrete implementation stages, each producing a fully tested and hardware-validated snapshot before the next stage began. This approach kept debugging tractable and ensured that a working fallback always existed.

The first functional stage established the framed protocol layer. A standalone QNX test program sent START, STOP, and STATUS commands as framed packets and validated the STM32 responses against expected reply types. This stage confirmed that CRC calculation matched across both platforms and that the parser handled all three request/reply flows correctly under real hardware conditions.

The second stage introduced the message-passing motor_ctrl server. UART ownership was moved inside motor_ctrl, and a demo_client was built to interact with it through MsgSend and MsgReply. This stage validated the complete synchronous control path: demo_client sends a typed request structure, motor_ctrl translates it into a UART packet, waits for the STM32 response, and returns a structured reply to the client. The client never accesses /dev/ser1 directly.

The third stage added alarm_mgr and introduced asynchronous fault notification. When motor_ctrl detects a fault condition, it sends a pulse to alarm_mgr using MsgSendPulse. Three fault conditions are handled: UART timeout, CRC or protocol mismatch, and an explicit FAULT packet from the STM32. alarm_mgr receives these pulses independently of any pending client request and logs the fault code and value to the terminal.

The fourth and most complex stage added continuous heartbeat monitoring. motor_ctrl now runs two background threads in addition to its main MsgReceive dispatch loop. A dedicated UART RX thread reads incoming bytes continuously and feeds them to the packet parser, updating a shared heartbeat timestamp each time a valid HEARTBEAT packet arrives. A separate heartbeat monitor thread wakes every 200ms, checks the elapsed time since the last heartbeat, and sends a PULSE_FAULT_HEARTBEAT_MISS to alarm_mgr if no heartbeat has been seen for more than two seconds. Client requests issued after a heartbeat miss are rejected with a HEARTBEAT_MISS status code rather than attempting a UART transaction against a disconnected or faulted device.

5. Tools and Technology

Component	Specification / Version	Role
Raspberry Pi 4B	4GB RAM, AArch64, QNX Neutrino 8.0.0	QNX RTOS controller
STM32 Nucleo-F401RE	ARM Cortex-M4, 84MHz, USART1	HIL plant simulator
QNX SDP 8.0	Momentics IDE, GCC cross-compiler	Build and deploy toolchain
STM32CubeIDE 1.14+	HAL, USART1 interrupt driver	STM32 firmware development
UART Link	115200 baud, 3.3V, PA9/PA10	Physical communication layer
QNX IPC APIs	name_attach, MsgSend, MsgSendPulse	Inter-process communication
CRC8-MAXIM	Polynomial 0x8C, reflected LSB-first	Packet integrity verification

Cross-compilation for the QNX AArch64 target was handled entirely within QNX Momentics IDE. Binaries were deployed to the Raspberry Pi using SCP over SSH, with the Pi's network interface configured manually after each boot using a fixed IP address of 192.168.1.50. The QNX filesystem is read-only at the root level, so all runtime binaries and writable state were placed in the /var RAM filesystem.

6. Intermediate Results and Testing

All four implementation stages were hardware-validated on the actual QNX and STM32 setup on 2026-04-14. The following outcomes were directly observed during validation testing.

Protocol layer validation: slice2_test start transmitted the packet AA 01 00 C4 and received AA 05 00 FF as the ACK response. The STATUS_REQ command returned AA 07 01 01 E0, with the data byte correctly indicating RUNNING state. The STOP command returned a clean ACK and the LED turned off. CRC matched on both sides without modification.

```
Command Prompt - ssh -i C:\  X  Windows PowerShell  X  +  v
409623    2  usr/sbin/qconn          10r RECEIVE          1
417816    1  usr/sbin/qconn          10r SIGWAITINFO
417816    2  usr/sbin/qconn          10r RECEIVE          1
# chmod +x /var/slice2_test
# ls -la /var/slice2_test
-rwxr-xr-x  1 root root 25488 1970-01-01 01:10 /var/slice2_test
# slay -f cat
# /var/slice2_test start
=== Slice 2 Protocol Test ===
Port: /dev/ser1
Command: start (0x01)

TX [4 bytes]: AA 01 00 C4

# /var/slice2_test start
=== Slice 2 Protocol Test ===
Port: /dev/ser1
Command: start (0x01)

TX [4 bytes]: AA 01 00 C4
RX: AA 05 00 FF  [VALID]
  Response CMD: 0x05 (ACK)

*** SLICE 2 TEST PASSED ***
# /var/slice2_test start
=== Slice 2 Protocol Test ===
Port: /dev/ser1
Command: start (0x01)

TX [4 bytes]: AA 01 00 C4
RX: AA 05 00 FF  [VALID]
  Response CMD: 0x05 (ACK)

*** SLICE 2 TEST PASSED ***
```

Message-passing layer validation: demo_client start returned status OK with response command 0x05 and the STM32 LED turned on. demo_client status returned OK with STATUS_RSP and reported motor state RUNNING. demo_client stop returned OK with ACK. The full synchronous control path through QNX message passing was confirmed working end to end.

CLIENT:-

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shukl>ssh -i %USERPROFILE%\ssh\id_ed25519 root@192.168.1.50
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client status
Reply status: OK (0)
Response cmd: 0x07
Data len: 1
Motor state: RUNNING
# /var/demo_client stop
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client status
Reply status: OK (0)
Response cmd: 0x07
Data len: 1
Motor state: RUNNING
# /var/demo_client stop
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# |
```

SERVER (MOTOR CONTROL):-

```
# chmod +x /var/demo_client
# ls -la /var/motor_ctrl /var/demo_client
-rwxr-xr-x 1 root root 17464 1970-01-01 02:49 /var/demo_client
-rwxr-xr-x 1 root root 27536 1970-01-01 02:49 /var/motor_ctrl
# slay -f cat
# /var/motor_ctrl
motor_ctrl ready on name 'pump_motor_ctrl'
UART port: /dev/ser1
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 03 00 55
RX byte: AA
RX byte: 07
RX byte: 01
RX byte: 01
RX byte: E0
TX [4 bytes]: AA 02 00 91
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 03 00 55
RX byte: AA
RX byte: 07
RX byte: 01
RX byte: 01
RX byte: E0
TX [4 bytes]: AA 02 00 91
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
```


Fault notification validation: after the STM32 was placed into FAULT state via the user button, a subsequent demo_client start returned REMOTE_FAULT (-4). Simultaneously, alarm_mgr printed REMOTE_FAULT (code=3 value=1), confirming that the pulse was delivered independently of the client reply. motor_ctrl logged STM32 reported FAULT packet and alarm pulse sent: code=3 value=1.

ALARM MANAGER:-

```
# slay -f motor_ctrl
# slay -f demo_client
# slay -f alarm_mgr
# rm /var/motor_ctrl
# rm /var/demo_client
# ls -la /var/alarm_mgr /var/motor_ctrl /var/demo_client
-rwxr-xr-x  1 root root 12832 1970-01-01 03:50 /var/alarm_mgr
-rw-r--r--  1 root root 19432 1970-01-01 03:56 /var/demo_client
-rw-r--r--  1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# chmod +x /var/motor_ctrl /var/demo_client
# ls -la /var/alarm_mgr /var/motor_ctrl /var/demo_client
-rwxr-xr-x  1 root root 12832 1970-01-01 03:50 /var/alarm_mgr
-rwxr-xr-x  1 root root 19432 1970-01-01 03:56 /var/demo_client
-rwxr-xr-x  1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# /var/alarm_mgr
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: REMOTE_FAULT (code=3 value=1)
```

SERVER (MOTOR CONTROL):-

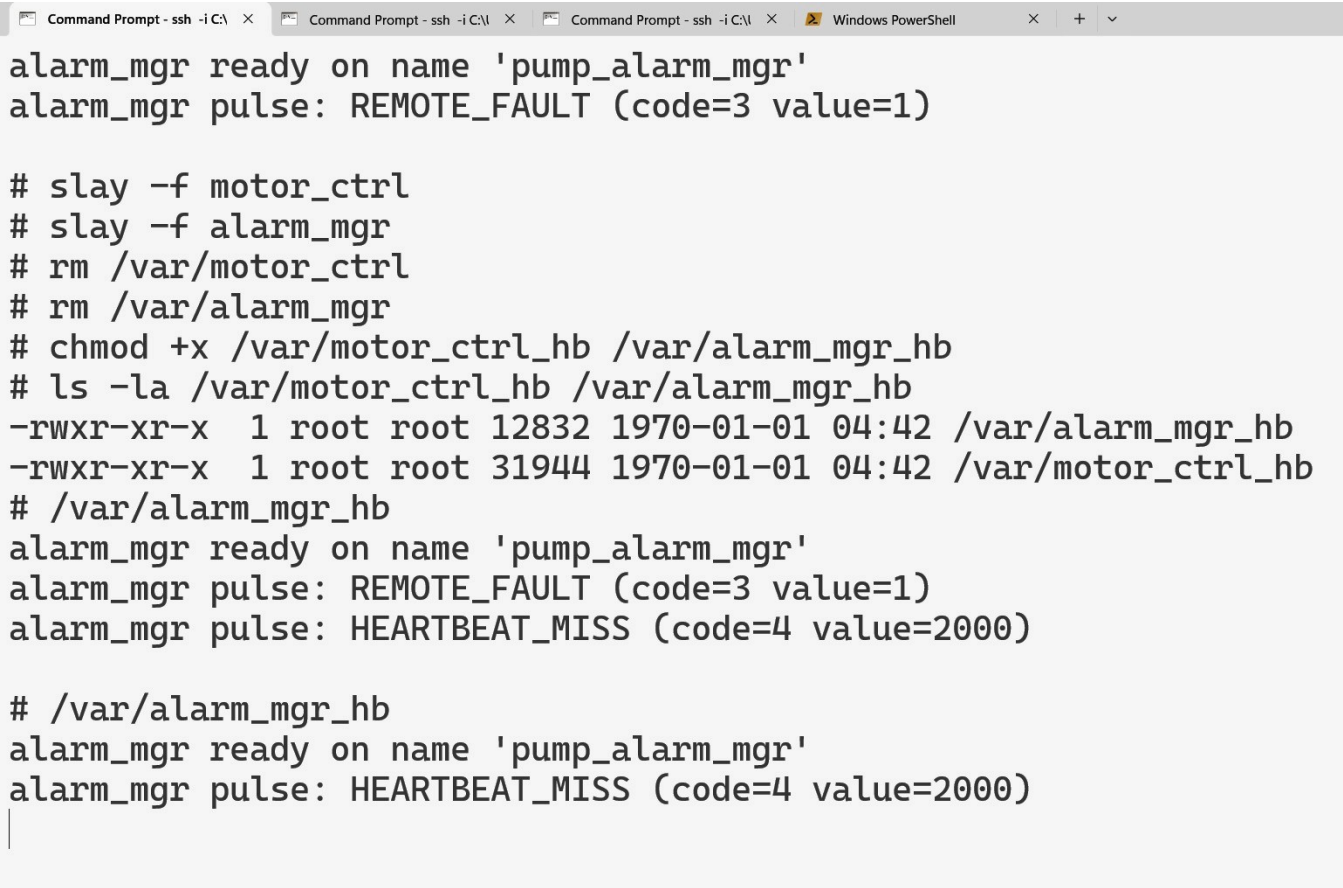
```
# /var/motor_ctrl
motor_ctrl ready on name 'pump_motor_ctrl'
UART port: /dev/ser1
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 06
RX byte: 00
RX byte: AA
STM32 reported FAULT packet
alarm pulse sent: code=3 value=1
```

CLIENT:-

```
# ls -la /var/alarm_mgr /var/motor_ctrl /var/demo_client
-rwxr-xr-x  1 root root 12832 1970-01-01 03:50 /var/alarm_mgr
-rwxr-xr-x  1 root root 19432 1970-01-01 03:56 /var/demo_client
-rwxr-xr-x  1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: REMOTE_FAULT (-4)
Response cmd: 0x00
Data len: 0
# |
```

Heartbeat detection validation: with the motor running and heartbeats flowing, motor_ctrl tracked heartbeat timestamps correctly in the background. After the UART connection was interrupted, alarm_mgr printed HEARTBEAT_MISS (code=4 value=2000) within approximately two seconds of the last received heartbeat. Subsequent client requests returned status code -6, correctly reflecting the degraded system state.

ALARM MANAGER:-



```
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: REMOTE_FAULT (code=3 value=1)

# slay -f motor_ctrl
# slay -f alarm_mgr
# rm /var/motor_ctrl
# rm /var/alarm_mgr
# chmod +x /var/motor_ctrl_hb /var/alarm_mgr_hb
# ls -la /var/motor_ctrl_hb /var/alarm_mgr_hb
-rwxr-xr-x  1 root root 12832 1970-01-01 04:42 /var/alarm_mgr_hb
-rwxr-xr-x  1 root root 31944 1970-01-01 04:42 /var/motor_ctrl_hb
# /var/alarm_mgr_hb
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: REMOTE_FAULT (code=3 value=1)
alarm_mgr pulse: HEARTBEAT_MISS (code=4 value=2000)

# /var/alarm_mgr_hb
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: HEARTBEAT_MISS (code=4 value=2000)
|
```

SERVER (MOTOR CONTROL):-

```
Command Prompt - ssh -i C:\ \ x  Command Prompt - ssh -i C:\ \ x  Command Prompt - ssh -i C:\ \ x  Windows PowerShell x + v
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
RX byte: AA
RX byte: 04
RX byte: 00
RX byte: 3B
alarm pulse sent: code=4 value=2000
|
```

CLIENT:-

```
Command Prompt - ssh -i C:\ \ x  Command Prompt - ssh -i C:\ \ x  Command Prompt - ssh -i C:\ \ x  Windows PowerShell x + v
-rwxr-xr-x 1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: REMOTE_FAULT (-4)
Response cmd: 0x00
Data len: 0
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: UNKNOWN (-6)
Response cmd: 0x00
Data len: 0
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client status
Reply status: UNKNOWN (-6)
Response cmd: 0x00
Data len: 0
# /var/demo_client stop
Reply status: UNKNOWN (-6)
Response cmd: 0x00
Data len: 0
```

7. Problem Solving and Debugging

Several issues were encountered and resolved during hardware bring-up that are worth documenting both for their debugging value and as evidence of the iterative engineering process.

During initial protocol testing, the STM32 responded to the first START packet by entering FAULT state and blinking the LED rather than returning an ACK. The UART transmission was confirmed working because the STM32 clearly reacted. Investigation of the STM32 state machine revealed that when `motor_state` was already FAULT or RUNNING from a previous session, the `handle_packet` function for `CMD_MOTOR_START` returned a FAULT packet rather than proceeding to the ACK path. The fix was to reset the STM32 to IDLE state before each test, and a permanent fix was to add an explicit state check at the start of the START handler. The `STATUS_RSP` response was also originally ambiguous because an early design overloaded the ACK command for all responses. A dedicated `CMD_STATUS_RSP` command was introduced to make response matching unambiguous. The QNX write path originally assumed that a single `write()` call would transmit the complete packet. On the QNX serial driver this is not guaranteed. A `write_full` helper function was introduced that loops until all bytes are confirmed written, which eliminated intermittent partial-packet transmission issues.

STM32 UART packet handling was originally executed directly inside the `HAL_UART_RxCpltCallback` interrupt handler. This caused timing issues when the callback attempted to call `HAL_UART_Transmit` while an ongoing receive was still in progress on the same peripheral. Moving all command execution into the main loop and keeping the interrupt handler strictly as a byte feeder into the parser resolved this interference.

During deployment, a naming inconsistency between the Momentics binary output names and the expected filenames on the Pi caused confusion where old binaries remained in place after SCP appeared to succeed. The root cause was that running processes held file locks. Explicitly running `slay` followed by `rm` before each SCP transfer resolved this reliably.

The heartbeat monitor thread required careful mutex discipline because the UART RX thread and the heartbeat monitor thread both access the `last_heartbeat_ms` timestamp. A shared `pthread_mutex` protects all reads and writes to the controller state structure. The condition variable used in the main dispatch loop allows the heartbeat monitor to signal the dispatch thread when a response arrives, avoiding busy-wait polling during UART transactions.

8. Limitations and Future Work

This project is a functional prototype and several limitations are worth stating clearly. The system handles one client request at a time. There is no queuing mechanism for concurrent clients, and multi-client scenarios have not been tested or designed for. This is acceptable for a demonstration system but would need to be addressed in any production design.

Automatic recovery after a heartbeat miss is not implemented. Once the `heartbeat_missed` flag is set, client requests fail with `HEARTBEAT_MISS` indefinitely until `motor_ctrl` is restarted. A practical system would need a supervised recovery path, potentially including a `MOTOR_STOP` command to the STM32 followed by a reconnection handshake.

The QNX Adaptive Partitioning Scheduler (APS) was not configured in this implementation phase. APS configuration and CPU budget enforcement across `motor_ctrl`, `alarm_mgr`, and a `cpu_hog` stress process remains a planned extension. Similarly, tracer-based interrupt latency measurement and regulatory-grade timing evidence generation were out of scope for this phase.

The project also does not implement the QNX secpol capability-based security model or the QNX Hypervisor for Safety virtualisation layer. These are documented in the architectural design but were explicitly deferred given the project timeline and the prototype nature of the work.

9. Conclusion

This project has successfully delivered a hardware-validated QNX-based infusion pump controller prototype across four incremental implementation stages. The protocol layer, the message-passing control server, the asynchronous alarm notification system, and the continuous heartbeat monitoring mechanism have all been verified on actual hardware. The system demonstrates real hardware interaction, a CRC-validated framing protocol, QNX-native synchronous IPC through `MsgSend` and `MsgReply`, asynchronous fault notification through pulses, and autonomous disconnect detection through background threading.

The design choices were driven by a combination of technical correctness and practical schedule discipline. The decision to use a message-passing server rather than a custom resource manager reduced implementation complexity without sacrificing the QNX-native IPC story. The incremental slice approach ensured that a working and demonstrable system existed at every stage of development, which significantly reduced debugging risk.

The project is best characterised as a safety-informed embedded prototype rather than a certified medical device implementation. It draws on design principles from safety-critical systems engineering and demonstrates their application on a real RTOS platform with real

hardware, within the honest constraints of an academic timeline. The remaining planned extensions, including APS partitioning and tracelogger timing analysis, represent natural next steps that build directly on the validated foundation established in this review phase.